

Подготовка к экзамену

Дисциплина: Нереляционные базы данных

Ответы на экзаменационные вопросы

Составлено на основе материалов курса:

- Лекции.pdf (презентации лекций)
- Базы_данных.pdf (учебное пособие, Фомин М.М., 2024)
- БД_Основные_термины.pdf (словарь терминов)

Вопрос 1

Концепция баз данных

Под концепцией базы данных понимается совокупность основополагающих принципов, по которым должны быть построены базы данных. Эти принципы были выработаны в середине 60-х годов XX века.

Основные концепции БД:

1. Отчуждение данных от программ. Данные должны быть отделены от программ: структура данных не должна зависеть от программы и наоборот. База данных с данными о студентах используется в разных программах — начисление стипендии, учёт успеваемости и т.д. Структура и наполнение этой базы не зависят ни от одной из них.
2. Хранение описания данных вместе с самими данными. Любой пользователь (человек или программа) должен иметь возможность узнать, какая информация хранится в БД, какова её структура и типы данных. Такие служебные данные (метаданные) хранятся в самой БД — это упрощает реализацию концепции отчуждения.
3. Отчуждение данных от носителей. Пользователь не должен знать, где физически хранится информация БД (на дисках, лентах, серверах на другом континенте). Он получает данные, не зная их расположения.
4. Поддержание БД в согласованном (целостном) состоянии. Согласованность БД — соответствие информации её внутренней логике, структуре и явно заданным правилам предметной области. Одна из важнейших задач БД — поддержание в любой момент взаимной непротиворечивости и точности хранящихся данных.
5. Защита информации. Это широкое понятие, охватывающее защиту от сбоев аппаратуры и программ, от злонамеренного или незлонамеренного искажения данных, от ошибок пользователей.
6. Поддержка многозадачной (многопользовательской) работы. Многочисленные пользователи (люди или программы) будут работать с данными БД одновременно. БД должна поддерживать многопользовательский режим, обеспечивая постоянный эффективный доступ к информации.

✎ Источник: *Лекции.pdf* — тема «Концепции баз данных», слайды «Концепции баз данных»; Учебное пособие «Базы данных.pdf» — раздел 1.1.3 «Концепции БД», стр. 10–11

Вопрос 2

Обеспечение живучести и отказоустойчивости. Копирование и репликация

Обеспечение живучести и отказоустойчивости СУБД строится на механизмах копирования и репликации данных.

Копирование базы данных осуществляется несколькими способами:

— Средствами операционной системы: делается копия файлов СУБД. На время копирования СУБД необходимо остановить, иначе можно получить несогласованную копию.

— Копирование устройств: аппаратными средствами системы хранения данных (СХД) создаётся копия устройства.

— Копирование специальными программами (средствами СУБД): создаётся согласованная копия базы данных — например, снимок всей БД с последующим копированием.

Репликация (replication) — это процесс поддержания в синхронном состоянии копий одних и тех же данных на нескольких узлах сети. Репликация решает три задачи: горизонтальное масштабирование серверов чтения, повышение надёжности и доступности системы, а также хранение данных географически ближе к пользователям.

Механизмы репликации строятся на использовании журналов изменений БД. Схема репликации: на основном сервере формируются журналы изменений, которые передаются на репликационный сервер и далее на серверы-реплики. На серверах-репликах применяются журналы, воспроизводя изменения.

Существуют следующие типы репликации: поблочная (WAL), построчная, логическая (репликация транзакций) и репликация моментальных снимков.

Суть репликации: изменения данных на мастере повторяются на репликах, но не наоборот. Запросы на изменение (INSERT, UPDATE, DELETE) выполняются только на мастере, а запросы на чтение (SELECT) — как на мастере, так и на репликах.

✎ Источник: Лекции.pdf — тема «Копирование базы данных» и «Репликация БД (общая схема)»; Учебное пособие «Базы данных.pdf» — раздел 2.2.11 «Копирование данных и резервные копии БД», стр. 43–45; раздел 2.2.12 «Репликация данных», стр. 45–48; БД_Основные термины.pdf — определение «Репликация», стр. 5

Вопрос 3

Нереляционные модели данных. Примеры реализации

До появления реляционной модели данных использовались иерархическая и сетевая модели.

Иерархическая модель данных. Информация хранится в структуре, напоминающей родословное дерево. Каждый «родитель» может иметь множество «детей», но каждый «ребёнок» имеет только одного «родителя» (отношение «один ко многим»). Примеры: файловая система компьютера, реестр Windows, СУБД IMS от IBM (1968). Недостатки: жёсткость структуры, сложность перестройки, избыточность данных при нескольких родителях.

Сетевая модель данных. Расширение иерархической модели: один родительский элемент может иметь множество дочерних, а дочерний — несколько родительских. Такое переплетение называется «множеством». Доступ к данным не обязательно начинается с корневого элемента. Наиболее популярная реализация — СУБД Adabas. Недостаток: все связи, структуры записей и таблиц должны быть заранее определены.

Постреляционная модель — реляционная модель, допускающая многозначные поля (атрибуты), т.е. само поле может быть таблицей.

Многомерная модель — узкоспециализированная модель для хранения данных в виде многомерного массива (гиперкуба). Используется в системах OLAP. Операции: срез (фиксация измерений), вращение (перестановка осей), агрегация/детализация (переход к более общему или детальному представлению).

NoSQL — термин, обозначающий подходы к реализации хранилищ данных, существенно отличающихся от реляционных СУБД. Основные характеристики: не используется SQL, неструктурированные (schemaless), данные в виде агрегатов, слабые ACID-свойства, распределённые системы. Примеры: MongoDB (документноориентированная), Redis, Cassandra, HBase.

✎ *Источник: Лекции.pdf — темы «Иерархическая модель данных», «Сетевая модель данных», «NoSQL»; Учебное пособие «Базы данных.pdf» — раздел 1.1.4 «Первые базы данных», стр. 11–13; БД_Основные термины.pdf — определения «Постреляционная модель», «Многомерная модель», стр. 4*

Вопрос 4

Ключи и индексы

Ключи и индексы — два разных инструмента в реляционной БД, решающих разные задачи.

Ключи служат для организации связей между таблицами. Типы ключей:

— Первичный ключ — поле, однозначно определяющее запись (экземпляр сущности). Значения в этом поле уникальны в пределах таблицы. Такой ключ может быть только один на таблицу. В реальных БД первичными используются суррогатные ключи.

— Вторичный ключ (кандидат) — поле с уникальным значением, которое могло бы быть первичным, но таковым не является.

— Внешний ключ — поле дочерней таблицы, содержащее значение первичного ключа родительской таблицы. При помощи внешних ключей осуществляются связи между таблицами.

Суррогатный ключ — специальный атрибут, не несущий информации об экземпляре сущности, а лишь идентифицирующий её. Его значение генерируется СУБД автоматически при добавлении записи. Суррогатные ключи идеальны, так как их значения уникальны, неизменны и не несут дополнительной семантики.

Индексы создаются для организации (ускорения) поиска и сортировки записей таблицы.

Индекс — объект БД (таблица), каждая запись которого содержит значение индекса и адрес записи индексируемой таблицы. Индексы строятся как правило в виде В-дерева.

Основные свойства индексов: ускоряют поиск и выборку; замедляют операции вставки, обновления и удаления; занимают место на диске; для маленьких таблиц не нужны; для временных таблиц не нужны; для часто обновляемых таблиц следует использовать минимальное количество индексов.

Синтаксис создания индекса: `CREATE [UNIQUE] INDEX имя_индекса ON имя_таблицы (имя_поля[, ...])`

✎ Источник: *Лекции.pdf* — тема «Ключи и индексы», «Что такое индекс», «Определение ключей»; *Учебное пособие «Базы данных.pdf»* — раздел 4.10.1 «Создание индексов», стр. 139–141; раздел 4.5.10 «Первичные и внешние ключи», стр. 119–120; *БД_Основные термины.pdf* — определения «Ключ», «Первичный ключ», «Внешний ключ», «Индекс», стр. 2

Вопрос 5

Реляционная модель данных. Таблицы, записи, поля, связи

Реляционная модель данных впервые описана в статье сотрудника фирмы IBM д-ра Эдгара Кодда, опубликованной 6 июня 1970 г. Кодд предложил использовать для обработки данных аппарат теории множеств (объединение, пересечение, разность, декартово произведение). Он показал, что любое представление данных может сводиться к совокупности двумерных таблиц — «отношений» (relation). Реляционной является БД, в которой все данные организованы в виде набора связанных двумерных таблиц, а все операции над данными сводятся к операциям реляционной алгебры.

Таблица в реляционной модели данных соответствует одной сущности предметной области и состоит из фиксированного числа полей, собранных в записи, каждая запись соответствует экземпляру сущности. У таблицы есть имя.

Запись (кортеж, строка таблицы) — данные об одном конкретном экземпляре объекта или события. Запись состоит из полей. У каждой записи есть ключ.

Поле (атрибут, столбец таблицы) — отдельная характеристика (свойство) сущности. Поле является контейнером для хранения данных об атрибутах объектов предметной области. У поля есть имя.

Связи между таблицами устанавливаются при помощи первичных и внешних ключей. Внешний ключ — поле дочерней таблицы, содержащее значение первичного ключа родительской таблицы. Типы связей: «один к одному» (1:1), «один ко многим» (1:M), «многие ко многим» (M:M) — последняя в реляционной БД не реализуется напрямую, а разрешается через промежуточную сущность.

Соответствие терминов: таблица = отношение (реляционная алгебра) = таблица (Excel); запись = кортеж = строка; поле = атрибут = столбец.

 Источник: Лекции.pdf — тема «Реляционная модель данных», «Таблицы, записи, поля», «Связи между таблицами»; Учебное пособие «Базы данных.pdf» — раздел 1.1.5 «Реляционная модель БД», стр. 13–15; БД_Основные термины.pdf — определения «Таблица», «Запись», «Поле», «Реляционная база данных», стр. 1–2, 4

Вопрос 6

Особенности языка SQL

SQL (structured query language — «язык структурированных запросов») — формальный непроцедурный декларативный язык программирования, применяемый для создания, модификации и управления данными в произвольной реляционной базе данных.

Ключевые особенности SQL:

1. Декларативность. SQL не является императивным (процедурным) языком. Вместо того чтобы указывать, как достичь результата, программист указывает, какого результата надо достичь. Всю работу выполняет оптимизатор СУБД.
2. Непроцедурность. В языке отсутствуют конструкции передачи управления: нет циклов, условных операторов, аппарата процедур, передачи параметров. SQL разрабатывался для непрограммистов.
3. Замкнутость. Входными данными для всех операторов языка SQL являются таблицы, выходными — тоже таблица. Это позволяет использовать выход одного оператора как входные данные для другого.
4. Зависимость от СУБД и независимость от платформы. SQL не может существовать вне ядра СУБД. Тип данных INTEGER всегда занимает 4 байта независимо от ОС.
5. Наличие стандартов. Первый стандарт SQL принят в 1986 г. (SQL86). Актуальный стандарт — SQL:2003 с редакцией 2008 г. Стандарт разбит на модули: SQL/Framework, SQL/Foundation, SQL/CLI (основа ODBC), SQL/PSM (процедурные расширения), SQL/Bindings, SQL/MM и др.
6. Диалекты. Каждая СУБД поддерживает собственный диалект SQL: PL/SQL (Oracle), Transact-SQL (MS SQL Server / Sybase), PL/pgSQL (PostgreSQL). Базовый синтаксис у всех совпадает.
7. Встроенные механизмы защиты данных. В SQL сразу встроены механизмы разграничения доступа — операторы GRANT и REVOKE.

✎ *Источник: Лекции.pdf — темы «Что такое SQL», «Непохожесть SQL», «Стандарты SQL», «Операторы SQL»; Учебное пособие «Базы данных.pdf» — разделы 3.2 «Концепции языка SQL», стр. 55–58; БД_Основные термины.pdf — определение «SQL», стр. 4*

Вопрос 7

Нереляционные базы данных и СУБД. Определение реляционной СУБД

Реляционная СУБД (РСУБД) — система управления базами данных, в которой реализован реляционный подход к организации данных. Согласно правилу Фомина, СУБД является реляционной, если в ней реализована полная поддержка языка SQL. В более строгом определении — если она удовлетворяет 12 правилам Кодда (1985 г.).

Все реляционные СУБД поддерживают язык SQL. К ведущим реляционным СУБД относятся: Oracle, IBM DB2, MS SQL Server, PostgreSQL, MySQL.

Нереляционные базы данных (NoSQL) — термин, обозначающий ряд подходов, направленных на реализацию хранилищ баз данных, имеющих существенные отличия от моделей, используемых в традиционных реляционных СУБД. Применяются к базам данных, где делается попытка решить проблемы масштабируемости и доступности за счёт атомарности и согласованности данных.

Характеристики NoSQL баз данных: 1) не используется SQL; 2) неструктурированные (schemaless) — структура данных не регламентирована, отслеживается в коде приложений; 3) представление данных в виде агрегатов; 4) слабые ACID-свойства; 5) распределённые системы без совместно используемых ресурсов.

Основное отличие NoSQL от реляционных: в NoSQL предпочтение отдаётся доступности и масштабируемости в ущерб строгой согласованности (теорема CAP). Вместо ACID используется BASE: базовая доступность (basic availability), гибкое состояние (soft state), согласованность в конечном счёте (eventual consistency).

 Источник: Лекции.pdf — темы «База данных и СУБД», «Одно правило Фомина», «NoSQL», «Характеристики NoSQL баз данных», «BASE вместо ACID»; Учебное пособие «Базы данных.pdf» — раздел 2.3 «Развитие реляционных СУБД», стр. 49–52

Вопрос 8

Нормализация баз данных

Нормализация — разбиение таблицы на две или более, обладающих лучшими свойствами при обновлении, включении и удалении данных.

Нормальные формы — это рекомендации по проектированию баз данных.

Цель нормализации сводится к получению такого проекта базы данных, в котором каждый факт (атрибут) появляется лишь в одном месте, т.е. исключена избыточность информации. Это делается не столько с целью экономии памяти, сколько для исключения возможной противоречивости хранимых данных и предсказуемости поведения системы во время эксплуатации.

Нормализация применялась преимущественно в прошлом веке при проектировании БД, когда проектирование рассматривалось независимо от конкретного приложения. Принцип нормализации непосредственно связан с принципом DRY (Don't Repeat Yourself): «Каждая часть данных должна иметь единственное, непротиворечивое и авторитетное представление в рамках системы».

Правило Фомина: «Если база данных нуждается в нормализации — значит она неправильно спроектирована. Если вы научитесь правильно проектировать базы данных, то нормализовать их не надо.»

В современном подходе корректно спроектированная БД на основе анализа бизнес-процессов и выделения сущностей автоматически приходит к нормализованному виду без применения формальных правил нормальных форм.

 Источник: *Лекции.pdf* — тема «Нормализация базы данных», «Правило нормализации Фомина»; *Учебное пособие «Базы данных.pdf»* — раздел 4.11 «Нормализация базы данных», стр. 143–144; раздел 4.3 «Основные цели проектирования БД», стр. 108; *БД_Основные термины.pdf* — определение «Нормализация», стр. 3

Вопрос 9

Основные функции СУБД

СУБД — совокупность программных, лингвистических и методических средств общего или специального назначения, обеспечивающих управление созданием и использованием БД.

Основные функции СУБД соответствуют концепциям БД:

- Управление данными во внешней памяти: обеспечивает отчуждение данных от носителей — пользователи не знают, где физически хранятся данные.
- Управление буферами оперативной памяти: СУБД поддерживает собственные буферы RAM для ускорения работы, поскольку размер БД существенно превышает объём оперативной памяти.
- Поддержка языков БД: обеспечивает отчуждение данных от программ. В реляционных СУБД стандартным языком является SQL. СУБД также поддерживает процедурные расширения (PL/SQL, Transact-SQL, PL/pgSQL).
- Ведение словаря БД: хранение описания данных вместе с самими данными. Словарь данных — набор таблиц, хранящих метаданные (структуру БД, объекты, пользователей и т.д.).
- Обеспечение согласованности БД: поддержание соответствия информации внутренней логике и правилам предметной области.
- Управление транзакциями: поддержание согласованности данных, изолированности и надёжности.
- Управление блокировками и клинчами: обеспечение корректной параллельной работы множества транзакций.
- Управление журналами изменений БД: обеспечивает защиту и восстановление данных после сбоев.
- Обеспечение безопасности БД: защита от сбоев и злонамеренных воздействий, разграничение доступа.

 *Источник: Лекции.pdf — тема «Основные функции СУБД»; Учебное пособие «Базы данных.pdf» — раздел 2.2 «Функции СУБД» (таблица 2.1), стр. 24*

Вопрос 10

Выделение сущностей и связей

Сущность — предмет, человек или событие, о котором собирается и хранится информация в БД. Каждой сущности соответствует отдельная таблица в реляционной БД.

Типы сущностей: объекты (предметы окружающего мира), субъекты (группы людей, роль), события (сущность, фиксирующая событие), документы (материальный носитель с информацией), справочники (перечни возможных типов сущностей или значений атрибутов).

Алгоритм определения сущностей. Сущности выделяются путём анализа бизнес-процессов: необходимо определить, с какими объектами работает пользователь, какие документы поступают на вход бизнес-процесса и какие формируются на выходе — это и есть сущности.

Верификация сущностей. После выделения сущностей их корректность проверяется по вопросам: отражает ли имя сущности суть объекта? Нет ли пересечения с другими сущностями? Имеются ли хотя бы два атрибута? Имеется ли у сущности хотя бы одна связь?

Выделение связей. Связь между сущностями — соответствие одному или нескольким экземплярам сущности А одного или нескольких экземпляров сущности В. Типы связей: один к одному (1:1), один ко многим (1:M), много ко многим (M:M). В реляционной БД все связи реализуются через первичные и внешние ключи только как «один ко многим».

Если при проектировании появляется связь «много ко многим» — это признак пропущенной сущности. Пример: авторы и книги — нужна промежуточная сущность «Библиография авторов».

 Источник: *Лекции.pdf* — темы «Выявление сущностей», «Определение сущностей», «Типы связей между сущностями»; *Учебное пособие «Базы данных.pdf»* — разделы 4.5.1–4.5.14, стр. 111–123; *БД_Основные термины.pdf* — определение «Сущность», стр. 1

Вопрос 11

Транзакции. Свойства транзакции

Транзакция — логическая единица работы, состоящая из одного или нескольких операторов SQL, которую СУБД рассматривает и обрабатывает как неделимое действие, переводящее БД из одного согласованного состояния в другое согласованное состояние. Допускается, что в процессе транзакции согласованность может нарушаться, но извне транзакции этого не видно.

Простой пример: перевод денег между счетами — это транзакция, включающая изменение состояния платёжного поручения, изменение сумм на счетах дебета и кредита, наращивание оборотов, формирование проводки. Все операции либо выполняются целиком, либо не выполняются вовсе.

Свойства транзакции (ACID):

— Atomic (атомарность): транзакция — неделимая единица, которая должна быть либо выполнена целиком, либо отменена. Не может быть выполнена лишь часть транзакции.

— Coordination (согласованность): смысл транзакции в том, чтобы база данных переходила из одного согласованного состояния в другое. БД всегда должна пребывать в согласованном состоянии.

— Insulativity (изолированность): каждая выполняемая транзакция не зависит от остальных. Промежуточные результаты работы транзакции не видны никому извне.

— Duration (надёжность): все результаты успешной транзакции наверняка сохраняются в базе данных. После завершения транзакции результаты не будут потеряны.

✎ Источник: Лекции.pdf — темы «Транзакция (простое определение)», «ACID, или свойства транзакции», «Транзакция (правильное определение)»; Учебное пособие «Базы данных.pdf» — раздел 2.2.5 «Управление транзакциями», стр. 30–34; БД_Основные термины.pdf — определение «Транзакция», стр. 3

Вопрос 12

Ключи как средство создания связей, первичные и внешние ключи

Связи между таблицами устанавливаются при помощи специальных полей — ключей.

Первичный ключ — поле, однозначно идентифицирующее запись в таблице (значение этого поля уникально в столбце). Используется для связи с другими таблицами.

Первичный ключ может быть только один на таблицу. На практике первичными используются суррогатные ключи — автоматически генерируемые значения (счётчики, последовательности), не несущие информации о сущности.

Внешний ключ — поле дочерней таблицы, которое содержит значение первичного ключа родительской таблицы. Это поле необходимо для создания связи между записями таблиц. Внешний ключ может принимать неопределённые значения (NULL) — например, в учебном плане есть дисциплина, но ещё не известно, кто будет её читать.

Родительская таблица — таблица, содержащая первичный ключ, значение которого записывается в поле внешнего ключа дочерней таблицы.

Дочерняя таблица — таблица, в которой присутствует внешний ключ, содержащий значение первичного ключа родительской таблицы.

Связь между родительской и дочерней таблицами — сопоставление записей по равенству значений внешнего и первичного ключей. Заполняется СУБД при добавлении записи (первичный ключ) и приложением путём копирования значения поля первичного ключа (внешний ключ).

Важный вопрос: что должно происходить при удалении целевой записи, на которую ссылается внешний ключ? Это разрушит базу данных. Поэтому первичные ключи используются только суррогатные (не меняются), а записи из базы данных лучше не удалять.

✎ Источник: *Лекции.pdf* — темы «Связи между таблицами», «Определение ключей», «Три основных вопроса про ключи и записи»; Учебное пособие «Базы данных.pdf» — раздел 4.5.10 «Первичные и внешние ключи», стр. 119–120; *БД_Основные термины.pdf* — определения «Первичный ключ», «Внешний ключ», «Родительская таблица», «Дочерняя таблица», стр. 2

Вопрос 13

Журналирование. Зачем нужны журналы транзакций?

Журналирование транзакций — функция СУБД, суть которой в сохранении информации, необходимой для восстановления базы данных в согласованное состояние в случае логических или физических сбоев.

В журнале изменений записывается следующая информация: индикаторы начала транзакции (порядковый номер, тип и время); пользователь, от имени которого проводились изменения; код транзакции на языке SQL; имя объекта, подвергшегося изменению; предыдущее и новое состояние объекта; индикаторы фиксации завершения транзакции.

Журналы транзакций нужны для решения трёх задач:

1. Восстановление данных после сбоев. При мягком сбое (потеря оперативной памяти) журнал используется для откатки незавершённых транзакций (журнал undo log) и наката завершённых (журнал redo log). При жёстком сбое (потеря данных на устройствах) — накат транзакций из архивного журнала (archived log) после загрузки резервной копии.
2. Репликация данных. Журналы изменений — основа механизма репликации: изменения с основного сервера через журналы передаются и воспроизводятся на серверах-репликах.
3. Анализ работы СУБД и действий пользователей. На основании журналов можно анализировать загрузку СУБД, работу пользователей, востребованность данных и оптимизировать работу системы.

СУБД Oracle использует оперативный журнал повтора (redo log), архивный журнал повтора (archive log) и оперативный журнал отката (undo log). Все изменения в БД происходят в рамках транзакций, поэтому журнал изменений часто называют журналом транзакций.

✎ Источник: Лекции.pdf — темы «Журналирование», «Зачем нужны журналы транзакций?»; Учебное пособие «Базы данных.pdf» — раздел 2.2.9 «Управление журналами изменений БД», стр. 39–40

Вопрос 14

Механизмы транзакций. Снапшот

Механизм снапшотов (snapshot) реализует принцип многоверсионного управления параллелизмом (MVCC — Multiversion Concurrency Control). Предполагается, что каждая транзакция видит так называемый снимок данных, т.е. то согласованное состояние (версию) базы данных, которое она имела на определённый момент времени.

Снапшот (от англ. snapshot — мгновенный снимок) — зафиксированное состояние данных на определённый момент времени.

Описание механизма: представим буферный кэш БД, состоящий из блоков памяти. Обращение к блокам идёт через «таблицу блоков» (информация о содержимом блоков и их адресах). В момент начала транзакции создаётся снапшот памяти — «основная таблица блоков» копируется, и новая транзакция (T1) обращается к данным через «таблицу блоков снапшота».

При добавлении данных: СУБД выделяет T1 новый блок, информация об этом блоке содержится только в таблице блоков снапшота T1. Остальные транзакции этот блок не видят.

При изменении данных: СУБД выделяет новый блок Q1 для изменённых данных. Старый блок Q остаётся в кэше и используется другими транзакциями. Таким образом достигается изолированность.

Успешное завершение (COMMIT): таблица блоков снапшота T1 копируется в основную таблицу блоков. Все результаты работы транзакции вступают в силу одномоментно.

Неудачное завершение (ROLLBACK): таблица снапшота T1 стирается, блоки освобождаются. Основная таблица блоков не меняется. Состояние БД восстанавливается на момент начала транзакции.

✎ Источник: *Лекции.pdf* — тема «Механизмы транзакций (snapshot)», «Окончание транзакции (успех)», «Окончание транзакции (неудача)»; *Учебное пособие «Базы данных.pdf»* — раздел 2.2.5 «Управление транзакциями», стр. 31–34

Вопрос 15

Восстановление СУБД после сбоев

Причины сбоев и последствия: пропало питание (утрачено содержимое оперативной памяти); ошибка пользователя или программного обеспечения (утрачены данные на устройстве долговременного хранения); глобальная катастрофа (потерян целиком ЦОД).

Мягкий сбой (потеря содержимого оперативной памяти). Данные на дисках остаются неповреждёнными. Восстановление:

1. Загрузка ближайшей точки согласованности. Точка согласованности — момент времени, когда содержимое ОЗУ согласованно с содержимым долговременной памяти (ORACLE делает точки согласованности не реже раза в 2 секунды).
2. Откат незавершённых транзакций по журналу undo log.
3. Накат завершённых транзакций из журнала redo log.
4. Повторный запуск транзакций, выполнявшихся в момент сбоя.

Жёсткий сбой (потеря данных на устройствах долговременного хранения).

Восстановление:

1. Загрузка данных с резервной копии (средствами ОС или средствами СУБД в однопользовательском режиме).
2. При необходимости правка журнала archived log (удаление транзакции, вызвавшей сбой).
3. Накат транзакций из журнала archived log.

При сбоях неизвестного происхождения (ошибки ПО, катастрофы) — единых правил нет.

При потере ЦОД — переход на резервный ЦОД с репликами серверов.

✎ Источник: Лекции.pdf — тема «Восстановление данных после сбоев (причины и последствия)», «Восстановление данных после сбоев (крах оперативной памяти)», «Восстановление данных после потери данных»; Учебное пособие «Базы данных.pdf» — раздел 2.2.10 «Обеспечение безопасности БД», стр. 40–43

Вопрос 16

Репликация БД (общая схема)

Репликация (replication) — механизм синхронизации содержимого нескольких копий БД. Это процесс, при котором данные копируются из одной БД в другую, и копии поддерживаются в актуальном и согласованном состоянии. При репликации изменения, сделанные в одной копии БД, распространяются в другие копии.

Общая схема репликации состоит из двух этапов:

1. Создание первоначальной копии. Копирование файловой системы (СУБД останавливается), копирование устройств или копирование специальными программами (снимок БД).
2. Распространение и применение изменений. На основном (мастер) сервере формируются журналы изменений данных. Они передаются на репликационный сервер, который распространяет журналы на серверы-реплики. На репликах производятся действия по применению журналов.

Информация об изменениях передаётся только в одну сторону: мастер → реплики. Поэтому на серверах-репликах доступны только запросы на чтение. Если позволить изменять данные на репликах, в системе возникнут многочисленные конфликты.

Терминология (из MS SQL Server): издатель (основной сервер), дистрибьютор (репликационный сервер), подписчик (сервер-реплика).

Возможна консолидация — слияние потоков журналов от нескольких серверов подразделений в единое хранилище. Это позволяет получать отчётность и аналитику по всей корпорации.

✎ Источник: *Лекции.pdf* — темы «Репликация. Второй шаг и рабочая схема», «Копирование базы данных»; *Учебное пособие «Базы данных.pdf»* — раздел 2.2.12 «Репликация данных», стр. 45–48; *БД_Основные термины.pdf* — определение «Репликация», «Тиражирование данных», стр. 5

Вопрос 17

Архитектура СУБД. Взаимодействие СУБД с клиентом

Архитектура СУБД рассматривается в двух аспектах: логическая (ANSI/SPARC) и физическая (файл-серверная, клиент-серверная, трёхзвенная).

Логическая архитектура ANSI/SPARC включает три уровня: внешний (пользовательский) — представление данных с точки зрения разных пользователей; промежуточный (концептуальный) — логическая структура БД, описание сущностей и связей; внутренний (физический) — физическое представление на носителях. Такая архитектура обеспечивает логическую и физическую независимость данных.

Файл-серверная архитектура: приложение на клиенте, БД на файловом сервере. Доступ через сеть путём пересылки файлов. Устаревшая технология (Paradox, dBase, FoxPro).

Клиент-серверная архитектура: система разбивается на клиентскую и серверную части. Клиент преобразует запросы в SQL, сервер выполняет запрос и возвращает результаты. Тонкий клиент (браузер) — реализуется на базе web-приложения. Толстый клиент — отдельное приложение с расширенной функциональностью. Примеры: Oracle, DB2, MS SQL Server, PostgreSQL, MySQL.

Трёхзвенная архитектура: три уровня — представление данных (клиент), бизнес-логика (серверы приложений), доступ к данным (серверы БД). Балансировщик нагрузки распределяет запросы между серверами приложений. Преимущества: простота модификации, расширения и интеграции; повышение безопасности; возможность работы тонкого клиента; независимость клиента от ОС.

Интерфейс ODBC (Open Database Connectivity) — посредник между приложением и СУБД; обеспечивает доступ из приложения к базам с различными СУБД через драйверы.

 *Источник: Лекции.pdf — темы «Клиент-серверная архитектура», «Трёхзвенная архитектура», «Что дает трёхзвенная архитектура»; Учебное пособие «Базы данных.pdf» — разделы 2.1.1–2.1.5, стр. 16–23; БД_Основные термины.pdf — определения «Клиент-серверная архитектура», «Трёхзвенная архитектура», «Тонкий клиент», «Интерфейс ODBC», стр. 6*

Вопрос 18

Поддержка транзакций в SQL

Транзакция в SQL — последовательность операторов, выделенная специальными командами управления транзакциями (группа TCL — Transaction Control Language).

Операторы управления транзакциями:

- START TRANSACTION — отмечает начало транзакции;
- SAVEPOINT точка_сохранения — отмечает промежуточную точку сохранения; позволяет откатить транзакцию не целиком, а до определённой точки;
- ROLLBACK [TO SAVEPOINT точка_сохранения] — откатывает все изменения текущей транзакции (или до точки сохранения);
- COMMIT — сохраняет все изменения текущей транзакции.

Любой оператор языка SQL, находящийся вне явных рамок транзакции, рассматривается СУБД как отдельная транзакция. Таким образом, все действия с данными БД проходят в контексте транзакционной концепции.

В SQL также поддерживаются уровни изоляции транзакций (TCL не включает их явно, но стандарт SQL их определяет): READ UNCOMMITTED, READ COMMITED, REPEATABLE READ, SERIALIZABLE. Более высокий уровень изоляции устраняет больше аномалий (грязное чтение, неповторяющееся чтение, фантомные записи), но снижает производительность.

В некоторых СУБД возможны вложенные транзакции — при неверном завершении вложенной транзакции откатываются все внешние.

✎ Источник: Лекции.pdf — тема «Поддержка транзакций в SQL»; Учебное пособие «Базы данных.pdf» — раздел 3.4.10 «Все операторы языка SQL» (группа TCL), стр. 87–88; раздел 2.2.8 «Уровни изоляции транзакций», стр. 38–39

Вопрос 20

Достижения реляционной алгебры

Реляционная алгебра — математический формализм, предложенный Эдгаром Коддом, основанный на теории множеств. Основные операции: объединение, пересечение, разность, декартово произведение.

Основные достижения реляционной алгебры:

1. Доказательство возможности представления любой структуры данных при помощи реляционной модели. Кодд показал, что любое представление данных может сводиться к совокупности двумерных таблиц-отношений.
2. Разработка принципов создания реляционных СУБД. Реляционная алгебра дала теоретическую основу для построения всех современных реляционных СУБД.
3. Разработка языка SQL. SQL является реализацией реляционной алгебры и реляционного исчисления в виде практического языка программирования.

Теорема Кодда: все три формализма — SQL, реляционная алгебра и реляционное исчисление — равнозначны. Любой запрос, выраженный на одном языке, может быть переформулирован на другом. Это позволяет транслировать запросы из SQL в реляционную алгебру для их анализа и оптимизации.

Соответствие терминов: отношение (реляционная алгебра) = таблица (БД/СУБД) = таблица (Excel); кортеж = запись = строка; атрибут = поле = столбец.

 Источник: Лекции.pdf — тема «Достижения реляционной алгебры»; Учебное пособие «Базы данных.pdf» — раздел 1.1.5 «Реляционная модель БД», стр. 15; раздел 3.2.5 «Равенство формализмов», стр. 57

Вопрос 21

Использование индексов и основные сведения об индексах

Индекс — объект БД, который создаётся для повышения производительности поиска данных. Индекс содержит записи, каждая из которых хранит два значения: значение индексируемого поля (ключ) и адрес соответствующей записи в таблице. Поиск по индексу организуется в виде В-дерева (сбалансированного дерева).

Синтаксис создания: `CREATE [UNIQUE] INDEX имя_индекса ON имя_таблицы (имя_поля[, ...])`

Что нужно помнить об индексах:

- Индексы создаются для ускорения поиска и выборки данных (SELECT).
- Индексы замедляют все операции, кроме выборки (INSERT, UPDATE, DELETE — при каждом изменении индекс пересчитывается).
- Индексы занимают место на диске.
- Для маленьких таблиц индексы не нужны.
- Для составного индекса важен порядок полей.
- Для временных таблиц индексы не нужны.
- Для таблиц, которые часто обновляются, используйте как можно меньше индексов.

Оптимизатор запросов в современных СУБД сам определяет наиболее эффективный способ использования индексов при выполнении запроса. Если для таблицы имеется подходящий индекс, оптимизатор будет его использовать.

 Источник: *Лекции.pdf* — тема «Что такое индекс», «Поиск по индексу (В-дерево)», «Создание индексов», «Что нужно помнить о индексах»; Учебное пособие «Базы данных.pdf» — раздел 4.10.1 «Создание индексов», стр. 139–141; БД_Основные термины.pdf — определение «Индекс», стр. 2

Вопрос 22

Группы операторов SQL и операторы, входящие в группы

Язык SQL содержит полный набор операций для любых действий с базой данных. Операторы разбиты на четыре группы:

1. DDL (Data Definition Language) — операторы определения данных:
 - CREATE — создаёт объект БД (таблицу, представление, индекс и т.д.);
 - ALTER — изменяет объект;
 - DROP — удаляет объект.
2. DML (Data Manipulation Language) — операторы манипуляции данными:
 - SELECT — выбирает данные, удовлетворяющие заданным условиям;
 - INSERT — добавляет новые данные;
 - UPDATE — изменяет существующие данные;
 - DELETE — удаляет данные.
3. DCL (Data Control Language) — операторы определения доступа к данным:
 - GRANT — предоставляет пользователю разрешения на определённые операции с объектом БД;
 - REVOKE — отзывает ранее выданные разрешения.
4. TCL (Transaction Control Language) — операторы управления транзакциями:
 - START TRANSACTION — отмечает начало транзакции;
 - COMMIT — сохраняет все изменения текущей транзакции;
 - ROLLBACK — откатывает все изменения текущей транзакции;
 - SAVEPOINT — делит транзакцию на более мелкие участки.

Вход и выход операторов языка SQL — всегда таблица. Это обеспечивает замкнутость языка SQL, унаследованную от реляционной алгебры.

🔗 Источник: *Лекции.pdf* — тема «Операторы SQL»; Учебное пособие «*Базы данных.pdf*» — раздел 3.4.10 «Все операторы языка SQL», стр. 87–88

Вопрос 23

Пользователи, роли и разграничение прав доступа. Предопределённые роли пользователей СУБД

Любые действия в СУБД производятся от имени пользователя. В основе разграничения доступа лежит модель дискреционного управления доступом (discretionary access control) — разграничение доступа между поименованными субъектами (пользователями) и поименованными объектами (таблицы, представления, процедуры и т.д.).

Роль — именованный набор привилегий, который можно присвоить пользователям. Роли упрощают управление правами: вместо назначения прав каждому пользователю отдельно, права назначаются роли, а пользователям назначается роль.

Предопределённые роли пользователей СУБД (на примере Oracle):

— CONNECT — конечные пользователи. По умолчанию им разрешено только соединение с БД и выполнение запросов к данным. Работает с данными, не может изменять объекты БД.

— RESOURCE — привилегированные пользователи, обладающие правом создания собственных объектов в БД (таблиц, представлений, синонимов, хранимых процедур). Пользователь-владелец объекта обладает полным набором привилегий для управления этим объектом. Работает с объектами БД, создаёт, модифицирует и удаляет объекты.

— DBA — категория администраторов базы данных. Включает возможности обеих предыдущих категорий, а также возможность вводить (удалять) в систему (из системы) субъекты защиты или изменять их категорию. Работает с пользователями, их схемами, табличными пространствами и другими объектами СУБД.

Операторы разграничения доступа: GRANT предоставляет привилегии (SELECT, INSERT, DELETE, UPDATE, ALTER, EXECUTE и т.д.) пользователю или роли. REVOKE отзывает ранее выданные права. Опция WITH GRANT OPTION позволяет пользователю передавать полученные привилегии дальше.

✎ Источник: Лекции.pdf — темы «Роли пользователей СУБД», «Разграничение доступа в СУБД»; Учебное пособие «Базы данных.pdf» — раздел 4.7 «Разграничение доступа и создание ролей пользователей», стр. 131–134; БД_Основные термины.pdf — определения «Дискреционное управление доступом», «Субъект доступа», «Объект доступа», «Санкционированный доступ», стр. 6–7

Вопрос 24

Блокировки и клинчи. Разрешение клинчей

Блокировка записей — процесс предотвращения доступа к данным, изменяемым транзакцией, пока она не будет закончена. Как только транзакция начинает изменять блок памяти (создаётся копия блока Q1), доступ к исходному блоку Q ограничивается для записи. Транзакции, пытающиеся изменить заблокированный блок, переводятся в режим ожидания и продолжают работу после снятия блокировки.

Клинч (deadlock) — взаимная блокировка двух или более транзакций. Транзакция 1 ожидает освобождения блока, заблокированного транзакцией 2, а транзакция 2 ожидает освобождения блока, заблокированного транзакцией 1. Ни одна из них не может продолжиться.

Множественные клинчи (кольцевые блокировки): несколько транзакций образуют замкнутую цепочку взаимных блокировок. СУБД достаточно просто находит подобные кольца.

Разрешение клинчей: необходимо разорвать порочный круг, принудительно завершив одну из транзакций. После этого кольцо распадается и блокировки разрешаются. Вопрос — какую транзакцию завершать?

— Самую старую (работающую дольше всего): дадим доработать транзакциям, уже затратившим большое количество процессорного времени.

— Самую молодую (только что начавшую работу): старая транзакция, возможно, заиклилась и держит всех.

Политику удаления транзакций определяет администратор СУБД в зависимости от местных особенностей и анализа статистики выполнения транзакций.

✎ Источник: Лекции.pdf — темы «Блокировки и клинчи», «Множественные клинчи», «Как выйти из клинча»; Учебное пособие «Базы данных.pdf» — раздел 2.2.6 «Управление блокировками и клинчами», стр. 34–37; БД_Основные термины.pdf — определение «Блокировка записей», стр. 3

Вопрос 25

Удаление записей и согласованность. Нереляционные базы данных

Удаление записей и согласованность. При удалении записей следует внимательно относиться к ограничениям ссылочной целостности. Нельзя удалить запись, на которую ссылаются внешние ключи других таблиц — это разрушит базу данных (ссылочная целостность будет нарушена). Поэтому основной принцип: никогда не удаляйте из базы данных записи!

Если параметр ON DELETE CASCADE был указан при создании таблицы, то вместе с удалением записи из главной таблицы будут удалены и все ссылающиеся записи дочерних таблиц. Этот режим опасен при некорректном условии WHERE.

На практике вместо физического удаления записей принято использовать логическое удаление: добавляется поле-флаг (например, «активен», «удалён», «дата закрытия»), и записи помечаются как удалённые, но физически остаются в базе.

Нереляционные базы данных. NoSQL-базы данных используются в ситуациях, когда требования к масштабируемости и доступности важнее строгой согласованности. Согласованность типа BASE (базовая доступность, гибкое состояние, согласованность в конечном счёте) предполагает, что данные могут быть некоторое время рассогласованы, но в конечном счёте приходят к согласованию.

В нереляционных БД нет строгих транзакций с полными ACID-свойствами. Каждый запрос может гарантированно завершиться (успешно или безуспешно), но данные могут быть некоторое время в несогласованном состоянии.

 Источник: Лекции.pdf — тема «Три основных вопроса про ключи (вопрос 3)»; Учебное пособие «Базы данных.pdf» — раздел 3.4.18 «Удаление данных. Оператор DELETE», стр. 96–97; раздел 4.8.3 «Удаление данных», стр. 137; Лекции.pdf — тема «NoSQL», «BASE вместо ACID»

Вопрос 26

Разграничения доступа в СУБД

Разграничение доступа в СУБД строится на модели дискреционного управления доступом. Субъект доступа — пользователь или процесс; объект доступа — таблица, представление, хранимая процедура и т.д.

Идентификация — присвоение пользователю уникального идентификатора.

Аутентификация — проверка, что именно данный пользователь предъявляет идентификатор.

Оператор GRANT предоставляет привилегии:

GRANT {{ALL [PRIVILEGES] | привилегия[, ...]} | роль[, ...]} [ON объект_БД] [TO получатель[, ...]] [WITH GRANT OPTION] [WITH ADMIN OPTION]

WITH GRANT OPTION — позволяет пользователю передавать полученные привилегии другим. WITH ADMIN OPTION — даёт возможность назначать полученную роль другим пользователям.

Среди привилегий могут быть: SELECT, INSERT, DELETE, UPDATE, ALTER, EXECUTE и другие.

Оператор REVOKE отзывает привилегии: REVOKE [GRANT OPTION FOR] {ALL [PRIVILEGES] | привилегия[, ...]} ON объект_БД TO|FROM {пользователь | роль} [CASCADE]. Ключевое слово CASCADE отзывает права не только у конкретного пользователя, но и у всех, кому он их передал.

На практике разграничение доступа осуществляется через роли: права назначаются ролям, а роли — пользователям. Это упрощает управление при большом числе пользователей.

 Источник: Лекции.pdf — темы «Разграничение доступа в СУБД», «Создание пользователей и ролей», «Права роли»; Учебное пособие «Базы данных.pdf» — раздел 4.7 «Разграничение доступа и создание ролей пользователей», стр. 131–135; раздел 3.4.19 «Защита данных в SQL», стр. 97–98; БД_Основные термины.pdf — определения «Идентификация», «Аутентификация», «Правила разграничения доступа», стр. 6–7

Вопрос 27

Триггеры и их использование в СУБД

Триггер — хранимая процедура особого типа, которую пользователь не вызывает непосредственно, а исполнение которой инициируется СУБД при попытке изменения данных в таблице, с которой он связан. Триггеры применяются для обеспечения согласованности данных и реализации сложной бизнес-логики.

Триггер запускается сервером автоматически при попытке изменения данных. Все производимые им модификации данных рассматриваются как выполняемые в транзакции, в которой выполнено действие, вызвавшее срабатывание триггера. При ошибке в триггере транзакция откатывается.

Синтаксис: CREATE TRIGGER имя_триггера {BEFORE | AFTER} {DELETE | INSERT | UPDATE [OF столбец]} ON имя_таблицы [FOR EACH {ROW | STATEMENT}] [WHEN (условие)] блок_кода

BEFORE — выполняется до DML операции (может подменить действие). AFTER — выполняется после. FOR EACH ROW — один раз для каждой изменённой строки. FOR EACH STATEMENT — один раз для всего оператора.

Применение триггеров:

- Автоматическая генерация значений полей (например, заполнение суррогатного первичного ключа);
- Логирование при ограничении доступа;
- Сбор статистики и аудит;
- Предотвращение DML операций (запрет удаления/изменения);
- Реализация сложных ограничений согласованности данных;
- Оповещение других модулей об изменениях;
- Реализация бизнес-логики;
- Каскадные воздействия на связанные таблицы.

✎ Источник: Лекции.pdf — темы «Триггеры», «Зачем использовать триггеры», «Определение триггера», «Пример триггера»; Учебное пособие «Базы данных.pdf» — раздел 4.6.4 «Триггеры», стр. 127–131; БД_Основные термины.pdf — определение «Триггер», стр. 3

Вопрос 29

Ключи и атрибуты. Суррогатные ключи как идеальные первичные ключи

Атрибут — свойство сущности в предметной области. Его наименование должно быть уникальным для конкретного типа сущности. В реляционной БД атрибуты хранятся в полях таблиц. Совокупность значений атрибутов определяет конкретный экземпляр сущности.

Ключи — особые атрибуты, однозначно определяющие экземпляры сущностей.

Первичный ключ — атрибут (или суперпозиция атрибутов), однозначно определяющий запись таблицы. Значения первичного ключа уникальны и неизменны.

Почему реальные атрибуты не подходят на роль первичного ключа: номер паспорта может меняться при замене; табельный номер при слиянии компаний становится неуникальным; номер студенческого билета не уникален глобально. Практически любой реальный атрибут в какой-то ситуации может оказаться ненадёжным.

Суррогатный ключ — специальный атрибут, не несущий никакой информации об экземпляре сущности, а только идентифицирующий её. Его значение генерируется СУБД автоматически при добавлении записи (обычно это счётчики или последовательности — sequences).

Преимущества суррогатных ключей: значения уникальны; значения неизменны на протяжении всего жизненного цикла записи; не зависят от предметной области; суррогатный первичный ключ группы уникален во всём вузе и на все времена. Если первичные ключи суррогатные, то меняться они не будут, и связи в базе данных не разрушатся при изменении реальных атрибутов.

 Источник: Лекции.pdf — тема «Определение ключей»: «Первичным ключом может быть только суррогатный ключ»; Учебное пособие «Базы данных.pdf» — раздел 4.5.6 «Первичный ключ – важный атрибут», стр. 116–117; раздел 4.5.10 «Первичные и внешние ключи», стр. 119–120; БД_Основные термины.pdf — определение «Первичный ключ», «Вторичный ключ», стр. 2

Вопрос 30

Стихийные угрозы информационной безопасности

Стихийные (незлонамеренные) угрозы информационной безопасности — это угрозы, которые не предполагают наличия злоумышленника. Сбой — утрата системой работоспособности.

Причины сбоев: неправильно написанное программное обеспечение (программные ошибки); неверные инструкции оператора (ошибки пользователя); отказ аппаратуры компьютера (аппаратные ошибки); внешние воздействия на аппаратный комплекс (катастрофы).

Мягкий сбой — аварийное выключение питания или сбой аппаратуры, не затрагивающий устройства долговременного хранения. Данные на дисках остаются неповреждёнными, но теряется содержимое буферов в ОЗУ. Восстановление — через точки согласованности и журналы.

Жёсткий сбой — разрушение данных на устройствах долговременного хранения. Может быть вызван отказом жёстких дисков или, гораздо чаще, некомпетентными или неосторожными действиями пользователя. Восстановление — через резервные копии и журналы.

Катастрофа — крупное неблагоприятное событие (авария, стихийное бедствие, пожар), влекущее полный отказ узла сети без возможности быстрого восстановления.

Единственный выход — переход на резервный ЦОД с репликами.

ЦОД (Центр Обработки Данных) создаются с целью минимизации рисков от катастроф.

Создание сетей ЦОДов — в первую очередь инженерно-строительная задача.

 Источник: Учебное пособие «Базы данных.pdf» — раздел 2.2.10 «Обеспечение безопасности БД» (подразделы «Мягкий сбой», «Жёсткий сбой», «Катастрофа»), стр. 40–43

Вопрос 31

BI системы. Путь данных от получения до анализа

BI (Business intelligence) — методы и инструменты для перевода необработанной информации в осмысленную, удобную форму для бизнес-анализа.

Великий путь данных в BI включает следующие этапы:

1. Информационный поиск (получение данных из различных источников).
2. ETL (Extract, Transform, Load — извлечение, преобразование, загрузка). Данные из оперативных систем (OLTP) очищаются, обогащаются, маппируются и загружаются в аналитическую систему.
3. Аналитическая обработка в реальном времени (OLAP). Данные укладываются в многомерные кубы. OLAP — технология подготовки суммарной (агрегированной) информации на основе больших массивов данных. Язык запросов к кубам — MDX (Multidimensional Expressions).
4. Витрина данных (Data Mart) — срез хранилища данных, представляющий собой массив тематической узконаправленной информации для пользователей одной рабочей группы или департамента.
5. Инструменты предупреждения об отклонениях от ожидаемых показателей.
6. Бизнес-аналитика — поиск закономерностей, тенденций и взаимозависимостей в данных.
7. Бизнес-отчётность — формирование отчётов для принятия решений.

Архитектурный принцип: «Много пользователей (много транзакций) → OLTP-БД (NoSQL) → обогащение, очистка, маппинг → свёртка, агрегирование → укладывание в OLAP-кубы → анализ, отчёты».

 Источник: Лекции.pdf — темы «Что такое BI», «Великий путь данных (BI)», «Этапы и части BI», «ETL», «OLAP», «Кубы в OLAP», «Витрина данных»

Вопрос 32

Злоумышленные угрозы информационной безопасности

Основные угрозы безопасности исходят от людей. Взламывают не системы, а людей! Практика показывает, что 90% случаев уязвимостей и утечек информации связаны с ошибками разработчиков и сопровождайцев или наличием сообщника злоумышленников в обслуживающем персонале.

Борьба со злонамеренными угрозами лежит в плоскости неформальных средств защиты: нормативных (законодательных), административных (организационных) и морально-этических.

Несанкционированный доступ (НСД) — доступ к информации, нарушающий правила разграничения доступа с использованием штатных средств вычислительной техники или автоматизированных систем.

Простые правила обеспечения безопасности (из учебного пособия):

- Настройка поддерживаемых систем: тщательная настройка всех модулей устраняет ненужные функции и уязвимости. Злоумышленники чаще всего предполагают стандартные настройки.
- Обновление систем: постоянное обновление позволяет устранить угрозы от известных вирусных атак и ошибки, исправленные в последних версиях ПО.
- Обучение персонала: обученный персонал быстро и грамотно реагирует на нештатные ситуации и попытки проникновения.
- Внедрение морально-этических средств: соблюдение норм коллективного поведения способствует защите информации.
- Распределение ролей и ограничение доступа к данным: разграничение доступа сводит к минимуму неумышленную порчу данных и злоумышленные попытки доступа.
- Создание всеохватывающей системы аудита: сохранение и анализ всех действий персонала позволяет понять причины сбоев.

 *Источник: Учебное пособие «Базы данных.pdf» — раздел 2.2.10 «Обеспечение безопасности БД» (подраздел «Злонамеренные угрозы и борьба с ними», «Простые правила обеспечения безопасности»), стр. 42–43; БД_Основные термины.pdf — определение «Несанкционированный доступ», стр. 6*

Вопрос 33

Хранимые (встроенные) процедуры в СУБД. Типы хранимых процедур

Хранимая процедура — объект БД, представляющий собой набор инструкций, который компилируется один раз и хранится на сервере в словаре БД. Хранимые процедуры пишутся на SQL или на расширении языка SQL (PL/SQL, T-SQL, PL/pgSQL) либо на внешних языках (Java, C#).

Хранимые процедуры могут: иметь входные и выходные параметры; использовать локальные переменные; выполнять числовые вычисления и операции над символьными данными; выполнять операции DDL и DML; включать циклы и условные ветвления.

Отличие от пользовательских функций: пользовательские функции можно использовать как выражение в SQL-запросе, а хранимые процедуры вызываются через CALL или EXECUTE.

Типы хранимых процедур:

- Процедуры обработки ошибок.
- Процедуры обеспечения целостности и согласованности БД.
- Процедуры реализации особо сложных запросов.
- Триггеры (особая разновидность — вызываются автоматически при изменении данных).

Синтаксис создания: CREATE {PROCEDURE | FUNCTION} имя_объекта ({[IN | OUT | INOUT] имя_параметра тип_данных}) [RETURNS тип_данных] [LANGUAGE {SQL | Java | C ...}] блок_кода

Хранимые процедуры и функции не являются альтернативой сложным SQL-запросам: по сравнению с «чистым» SQL любая процедурная альтернатива является худшим вариантом с точки зрения производительности.

🔗 Источник: Лекции.pdf — темы «Типы встроенных процедур», «Встроенные процедуры в СУБД»; Учебное пособие «Базы данных.pdf» — раздел 4.6.3 «Создание хранимых процедур», стр. 127

Вопрос 34

Простые действия для обеспечения безопасности IT-систем

Про обеспечение безопасности в учебном пособии «Базы данных» предлагаются следующие базовые правила, обязательное следование которым позволит устранить большую часть угроз безопасности ИС:

1. Настройка поддерживаемых систем. Тщательная настройка всех модулей системы позволяет убрать все ненужные функции ПО, что сужает количество уязвимостей. Злоумышленники чаще всего предполагают, что настройки системы стандартные, поэтому тщательная настройка устраняет многие угрозы.
2. Обновление поддерживаемых систем. Постоянное обновление и поддержка актуального состояния позволяют устранить ряд угроз от известных вирусных атак и избежать ошибок, исправленных в последних версиях ПО.
3. Обучение персонала (пользователей). Обученный персонал быстро и грамотно реагирует на нештатные ситуации, сбои и попытки проникновения. Это позволяет сократить время реакции администраторов.
4. Внедрение морально-этических средств. Соблюдение сложившихся норм и этических правил способствует защите информации, а нарушение приравнивается к несоблюдению правил поведения.
5. Распределение ролей и ограничение доступа к данным. Разграничение доступа персонала к данным позволяет свести к минимуму неумышленную порчу данных и злоумышленные попытки.
6. Создание программ проверки целостности. Постоянные проверки согласованности, непротиворечивости и достоверности информации обеспечивают своевременное выявление нештатных ситуаций.
7. Создание всеохватывающей системы аудита. Сохранение и анализ всех действий персонала даёт возможность понять причины сбоев и искажения данных.

 *Источник: Учебное пособие «Базы данных.pdf» — раздел 2.2.10 «Обеспечение безопасности БД» (подраздел «Простые правила обеспечения безопасности БД»), стр. 42–43*

Вопрос 35

Трёхзвенная архитектура и её преимущества

Трёхзвенная архитектура строится на разделении обработки данных на три физических уровня: слой представления данных (клиент), слой бизнес-логики (серверы приложений) и слой доступа к данным (серверы БД).

Каждый слой предоставляется в виде сервиса и обычно размещён на одном или нескольких выделенных серверах. Балансировщик нагрузки — аппаратная схема или сервер, распределяющий запросы к серверам приложений.

Преимущества трёхзвенной архитектуры:

- Простота модификации: изменение одного слоя не затрагивает другие.
- Простота расширения: горизонтальное масштабирование слоя бизнес-логики — просто добавить серверы.
- Простота интеграции: слой БД может быть разнородным (включать разные СУБД).
- Повышение безопасности: введение слоя бизнес-логики обеспечивает дополнительную защищённость данных.
- Возможность работы тонкого клиента (браузера): низкая стоимость внедрения клиента; простая поддержка; независимость клиента от ОС; доступность из любой точки мира.
- Отказоустойчивость: резервные серверы на каждом уровне принимают нагрузку при отказах.
- Возможность модернизации: отдельные звенья можно выводить из эксплуатации для обновления, не ухудшая общей производительности за счёт резервных серверов.

Недостатки: трудность проектирования; трудность отладки; эксплуатация требует высококвалифицированного персонала. Эти трудности быстро уменьшаются с развитием технологий разработки.

✎ Источник: Лекции.pdf — темы «Трёхзвенная архитектура», «Что дает трёхзвенная архитектура»; Учебное пособие «Базы данных.pdf» — раздел 2.1.5 «Трёхзвенная архитектура», стр. 21–23; БД_Основные термины.pdf — определение «Трёхзвенная архитектура СУБД», стр. 6

Вопрос 37

Документоориентированные СУБД на примере MongoDB

Документоориентированные СУБД — разновидность NoSQL-баз данных, где основной единицей хранения является документ (обычно в формате JSON или BSON). MongoDB — одна из самых популярных документоориентированных СУБД.

Основные характеристики NoSQL (применимые к MongoDB): не используется SQL; неструктурированные (schemaless) — структура данных не регламентирована, схема данных отслеживается в коде приложений; представление данных в виде агрегатов — сущности объединяются в объекты базы данных как удобно; слабые ACID-свойства; распределённые системы без совместно используемых ресурсов.

В курсе «Нереляционные базы данных» MongoDB рассматривается как практический пример реализации документоориентированной СУБД. Данная СУБД хранит данные в виде коллекций документов (аналог таблиц в реляционных БД), где каждый документ — аналог записи, но может иметь произвольную структуру, вложенные объекты и массивы.

Место NoSQL в информационных системах: NoSQL-базы применяются там, где объём данных и скорость их обработки важнее строгой согласованности. Примеры применения: системы с большим потоком транзакций, системы хранения неструктурированных данных, аналитические системы (первый этап пути данных — OLTP → ETL → OLAP).

Согласно концепции CAP-теоремы: в распределённой системе невозможно одновременно обеспечить более двух из трёх свойств: согласованность (consistency), доступность (availability), устойчивость к разделению (partition tolerance). NoSQL-системы, как правило, жертвуют строгой согласованностью ради доступности и устойчивости к разделению.

✎ Источник: Лекции.pdf — темы «NoSQL», «Характеристики NoSQL баз данных», «Теорема CAP», «SQL и NoSQL», «Много — много СУБД»; Учебное пособие «Базы данных.pdf» — раздел 2.3 «Развитие реляционных СУБД» (о NoSQL), стр. 51–52

Вопрос 38

Жизненный цикл информационной системы

Жизненный цикл информационной системы (ЖЦ ИС) — процесс функционирования системы с момента её создания до вывода из эксплуатации.

В начале 80-х годов прошлого века известный отечественный учёный В.В. Липаев предложил следующую схему жизненного цикла ИС:

1. Анализ требований — изучение предметной области, сбор информации о бизнес-процессах, определение ограничений и критериев успеха проекта.
2. Разработка требований — формирование спецификации: что должно быть реализовано, поведение системы, свойства и атрибуты, ограничения разработки.
3. Проектирование — разработка бизнес-процессов, определение ролей пользователей, определение сущностей, создание схемы сущностей и связей, техническое проектирование БД.
4. Разработка и тестирование — реализация проекта в коде, создание объектов БД, тестирование функциональности.
5. Внедрение — установка системы и ввод в эксплуатацию.
6. Эксплуатация — использование системы по назначению. Эксплуатация — часть жизненного цикла системы, на протяжении которой она используется по назначению. Включает процессы управления инцидентами, проблемами, конфигурациями, изменениями, мощностями и т.д. (ITIL/ITSM).
7. Вывод из эксплуатации — подготовка данных к переносу в новую ИС, архивация, консервация, обеспечение получения исторической отчётности.

Современный жизненный цикл — итеративный: система последовательно выходит версиями (Версия 1, Версия 2, Версия 3...), каждая из которых проходит этапы разработки, тестирования и внедрения.

 Источник: Лекции.pdf — темы «Жизненный цикл информационной системы», «Вывод из эксплуатации»; Учебное пособие «Базы данных.pdf» — раздел 4.1 «Исторический экскурс», стр. 104

Вопрос 39

Основные цели и основной результат проектирования БД

Проектирование БД — создание информационной модели предметной области на необходимом уровне абстракции в терминах сущностей (таблиц) и их экземпляров, связей между ними и правил согласованности данных.

Основные цели проектирования БД:

1. Обеспечение хранения в БД всей необходимой информации. Эта цель должна быть достигнута непременно. Изучайте предметную область, не упустите ни одной сущности и атрибута.
2. Обеспечение возможности получения данных по всем необходимым запросам. Все сущности должны быть связаны между собой так, чтобы одним запросом можно было достать любые данные, необходимые для ИС.
3. Сокращение избыточности и дублирования данных. Принцип DRY (Don't Repeat Yourself): «Каждая часть данных должна иметь единственное, непротиворечивое и авторитетное представление в рамках системы».
4. Обеспечение согласованности данных. Соблюдение ограничений, накладываемых на данные в БД в соответствии с предметной областью.

Основной результат проектирования БД — программа (скрипт) на языке SQL, который содержит:

- Операторы создания таблиц;
- Операторы создания представлений;
- Операторы создания индексов;
- Операторы создания ролей пользователей;
- Операторы создания хранимых процедур и других объектов БД.

Запустив этот скрипт в конкретной СУБД, мы получим полную схему всей БД, роли и механизмы разграничения прав пользователей, механизмы обеспечения согласованности, безопасности и быстродействия БД.

 *Источник: Лекции.pdf — темы «Основные цели проектирования БД», «Результат проектирования БД»; Учебное пособие «Базы данных.pdf» — разделы 4.3 «Основные цели проектирования БД», стр. 108; 4.4.2 «Результат проектирования БД», стр. 110*

Вопрос 41

Что такое NoSQL. Место NoSQL баз в информационных системах

NoSQL — термин, обозначающий ряд подходов, направленных на реализацию хранилищ баз данных, имеющих существенные отличия от моделей, используемых в традиционных реляционных СУБД. Применяется к базам данных, где делается попытка решить проблемы масштабируемости и доступности за счёт атомарности и согласованности данных.

Характеристики NoSQL: не используется SQL; неструктурированные (schemaless); данные в виде агрегатов; слабые ACID-свойства (BASE: Basically Available, Soft state, Eventually consistent); распределённые системы без совместно используемых ресурсов.

Место NoSQL в информационных системах. Принцип «Каждому своё»: много пользователей (много транзакций) → NoSQL-БД (для хранения оперативных данных) → обогащение, очистка, маппинг (ETL) → свёртка, агрегирование → укладывание в OLAP-кубы → анализ, отчёты.

NoSQL решает проблемы, с которыми плохо справляются реляционные СУБД при распределённом хранении: реляционные СУБД с полными ACID-свойствами при работе с несколькими серверами требуют сложных протоколов синхронизации (протокол двухфазной фиксации), что существенно снижает масштабируемость.

Однако NoSQL поддерживает весьма узкий круг задач, связанных с накоплением, очисткой и выборкой малосвязанных данных. В настоящее время большинство существующих СУБД являются реляционными — они удерживают лидирующие позиции при построении промышленных ИС.

✎ Источник: *Лекции.pdf* — темы «NoSQL», «Характеристики NoSQL баз данных», «Теорема CAP», «BASE вместо ACID», «Каждому своё»; Учебное пособие «Базы данных.pdf» — раздел 2.3 «Развитие реляционных СУБД» (о NoSQL), стр. 51–52

Вопрос 42

Администрирование СУБД. Основные функции администратора СУБД

Администратор базы данных (DBA, Database Administrator) — лицо или группа лиц, ответственная за развёртывание, бесперебойную эксплуатацию и эффективное использование СУБД.

Основные функции администратора СУБД (по рекомендациям Oracle):

- Установка и обновление программного обеспечения Oracle и прикладных средств.
- Распределение внешней памяти системы и планирование дальнейших потребностей в ней.
- Создание первичных структур хранения базы данных (табличных пространств).
- Регистрация пользователей и обеспечение защиты системы.
- Обеспечение соответствия лицензионному соглашению.
- Управление и мониторинг доступа пользователей к базе данных.
- Мониторинг и оптимизация производительности базы данных.
- Резервное копирование и восстановление базы данных.
- Связь с технической поддержкой.

Дополнительно (из основных функций администратора БД): участие в разработке проекта по созданию БД; обеспечение согласованности БД (копирование, репликация, восстановление); обучение пользователей; отслеживание трафика и сбоев; реорганизация БД с целью улучшения характеристик; реализация многопользовательского режима (защита, шифрование, разделение доступа).

Настройка размера буферов оперативной памяти — один из ключевых навыков DBA любой СУБД. Администратор СУБД также выбирает политику разрешения клинчей в зависимости от анализа статистики выполнения транзакций.

✎ Источник: *Лекции.pdf* — тема «Администрирование СУБД»; Учебное пособие «Базы данных.pdf» — раздел 5 «Администрирование БД» (заголовок), стр. 145; БД_Основные термины.pdf — определения «Администратор базы данных», «Администратор СУБД», «Основные функции администратора», стр. 5

Вопрос 43

Типы связей между сущностями и их реализация в реляционной СУБД

Связь между сущностями — соответствие одному или нескольким экземплярам сущности А одного или нескольких экземпляров сущности В.

Типы связей:

1. Один к одному (1:1). Одной записи родительской таблицы соответствует одна запись дочерней. Редкий тип связи. Пример: Физическое лицо — Паспорт. Реализация в реляционной СУБД: внешний ключ с ограничением уникальности (UNIQUE).
2. Один ко многим (1:M). Одной записи родительской таблицы соответствует несколько записей дочерней. Это основной тип связи в реляционной БД. Реализация: первичный ключ в родительской таблице + внешний ключ в дочерней (без ограничения уникальности).
3. Много ко многим (M:M). Один экземпляр родительской сущности соответствует нескольким экземплярам дочерней и наоборот. В реляционной БД НЕ существует прямой реализации связи «много ко многим» — наличие такой связи говорит о том, что пропущена какая-то значимая сущность. Именно она преобразует связь M:M в две связи 1:M. Пример: авторы и книги → промежуточная сущность «Библиография авторов».

В реляционной БД все связи между таблицами реализуются только через первичные и внешние ключи. Поэтому реально существуют только два типа связей: один к одному и один ко многим.

Что происходит при удалении записи родительской таблицы, на которую ссылается внешний ключ: база данных разваливается! Поэтому в реальных БД записи не удаляются, а помечаются как недействующие. При изменении первичного ключа — аналогичная проблема, поэтому используются только неизменяемые суррогатные первичные ключи.

✎ Источник: *Лекции.pdf* — темы *Типы связей между сущностями*, *Реализация связей много-много*; *Учебное пособие Базы данных.pdf* — разделы 4.5.9–4.5.12, стр. 118–122; *БД_Основные термины.pdf* — определения *Связь типа Один-к-одному*, *Связь типа Один-ко-многим*, *Связь типа Многие-ко-многим*, стр. 2

Вопрос 44

Секционирование и кластеризация таблиц

Секционирование (partitioning) — разделение хранимых объектов баз данных на отдельные части с отдельными параметрами физического хранения.

Применение секционирования: позволяет «обрезать» большие таблицы, разделив данные по смысловому признаку (например, по времени: документы за последний год / за предыдущий год; записи по активным / пассивным пользователям). При запросах к секционированной таблице СУБД может обращаться только к нужным секциям, минуя остальные, что существенно ускоряет выборку.

Примеры секционирования: по дате (документы за последний год, за предыдущий год, архивные); по значению ключа (записи активных и пассивных пользователей); по диапазону первичных ключей (в системах репликации у каждой БД подразделения — свой разрешённый диапазон ключей).

Кластеризация таблиц — попытка разместить рядом в одном физическом блоке данных те записи, доступ к которым осуществляется при помощи одинаковых значений ключа (индекса). При кластеризации записи с одинаковыми значениями ключа физически хранятся в одном или соседних блоках памяти.

Применение кластеризации: когда запросы к таблице часто выбирают группы записей с одинаковым значением какого-то поля, кластеризация по этому полю позволяет уменьшить количество обращений к дискам за счёт последовательного чтения блоков, а не случайного поиска.

Оба механизма — секционирование и кластеризация — являются инструментами обеспечения быстродействия БД и настраиваются на этапе физического проектирования.

 *Источник: Лекции.pdf — темы «Секционирование таблиц», «Кластеризация таблиц»; Учебное пособие «Базы данных.pdf» — раздел 4.10.2 «Кластеризация таблиц», стр. 141–142; раздел 4.10.3 «Секционирование таблиц», стр. 142–143*

Вопрос 46

Представления и их место в БД

Представление (view) — виртуальная (логическая) таблица, представляющая собой поименованный запрос (синоним к запросу), который будет подставлен как подзапрос при использовании представления. К представлению обращаются как к обычной таблице.

Представления хранятся в СУБД в виде скомпилированных запросов, которые динамически наполняют данными виртуальные таблицы в оперативной памяти. В отличие от таблиц, представления не занимают физическое пространство на диске.

Что дают представления:

1. Скрывают от прикладной программы сложность запросов и саму структуру таблиц БД.
2. Позволяют использовать запрещённые в обычных таблицах элементы, например, вычисляемые поля — значение всегда актуально, так как вычисляется в момент обращения.
3. Позволяют отделить прикладную схему представления данных от схемы хранения.
4. Обеспечивают ещё один уровень защиты данных: данные в представлении нельзя изменить по умолчанию; часть таблицы можно закрыть для пользователей через ограничивающее представление.
5. СУБД может применить к запросу представления оптимизацию или предварительную компиляцию, поэтому представления работают быстрее обычных запросов.

Синтаксис создания: `CREATE [RECURSIVE] VIEW имя_представления [(поле[, ...])] AS оператор_select [WITH [CASCADED|LOCAL] CHECK OPTION]`

Место представлений в архитектуре ANSI/SPARC: представления реализуют внешний (пользовательский) уровень архитектуры, обеспечивая независимость внешних программ от физической структуры хранения данных.

 Источник: *Лекции.pdf* — темы «Представления в СУБД», «Что дают представления»; Учебное пособие «Базы данных.pdf» — разделы 3.4.15 «Представления в языке SQL», стр. 92–94; 4.6.2 «Создание представлений», стр. 125–127; *БД_Основные термины.pdf* — нет явного определения, но структура СУБД как часть обеспечения безопасности

Вопрос 47

Согласованность БД. Типы согласованности

Согласованность (целостность) базы данных — соответствие имеющейся в БД информации её внутренней логике, структуре и всем явно заданным правилам предметной области. Каждое правило, налагающее ограничение на возможное состояние БД, называется ограничением согласованности.

Примеры правил: в неделе семь дней; вес детали должен быть положительным; количество знаков в телефонном номере не должно превышать 15; баланс банка должен сходиться; число проданных билетов не превышает число посадочных мест.

Типы согласованности БД:

— Сущностная целостность: каждая строка в таблице уникальна, идентифицируется первичным ключом.

— Доменная целостность: значения полей принадлежат допустимому домену (типу данных, диапазону значений).

— Ссылочная целостность: значения внешних ключей соответствуют значениям первичных ключей родительских таблиц.

— Пользовательская целостность: ограничения, определённые пользователем в соответствии с требованиями предметной области (бизнес-правила).

Механизмы обеспечения согласованности: ограничения NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK; транзакции (ACID); триггеры; хранимые процедуры.

В распределённых системах согласованность рассматривается по-другому. Теорема CAP: невозможно одновременно обеспечить строгую согласованность (Consistency), доступность (Availability) и устойчивость к разделению (Partition tolerance). BASE — альтернатива ACID: данные могут быть некоторое время рассогласованы, но приходят к согласованию через некоторое время (eventual consistency).

✎ Источник: *Лекции.pdf* — темы «Теорема CAP», «BASE вместо ACID»; Учебное пособие «Базы данных.pdf» — раздел 2.2.4 «Обеспечение согласованности БД», стр. 29; раздел 4.8 «Обеспечение согласованности БД», стр. 135–137; раздел 4.8.1 «Виды согласованности БД», стр. 135; *БД_Основные термины.pdf* — определение «Согласованность (целостность) базы данных», стр. 3

Вопрос 48

Что такое База Данных

База данных (БД) — организованная в соответствии с определёнными правилами и поддерживаемая в памяти компьютера совокупность данных, характеризующая актуальное состояние некоторой предметной области и используемая для удовлетворения информационных потребностей пользователей. (Когаловский М.Р. Энциклопедия технологий баз данных, 2002)

Предметная область — часть реального мира, о которой база данных хранит, собирает и анализирует информацию. В БД хранятся не все данные о предметной области, а только те, которые позволяют решать определённые задачи.

Под пользователями понимаются любые потребители информации — люди или программы. Обращение к БД стандартизировано использованием единого языка запросов (SQL для реляционных СУБД).

По своей сути любая БД — скрупулёзно построенное хранилище структурированных данных, интегрированное в более сложную надсистему — информационную систему.

Никогда не путайте термины «База данных» и «СУБД». База данных — это данные. СУБД — это программное обеспечение, которое управляет этими данными. «Среди непрофессионалов [...] возникает путаница при использовании терминов «база данных» и «система управления базами данных». [...] Мы будем строго разделять эти термины» (Кузнецов С.Д. Основы баз данных).

Данные без описания (без контекста) превращаются в цифровой шум. Поэтому описание данных (метаданные) должно храниться в самой БД — это реализует концепцию «Хранение описания данных вместе с самими данными».

 Источник: Лекции.pdf — тема «База данных, что это?», «База данных и СУБД»; Учебное пособие «Базы данных.pdf» — раздел 1.1.2 «Понятие БД», стр. 9–10; БД_Основные термины.pdf — определение «База данных (БД)», стр. 1